

Prosody to Music:
Dal testo poetico alla musica

Eisa Caprio
Rita Tiani

Università degli Studi Roma Tre
LM-32 Ingegneria Informatica
Corso Artificial Intelligence from Engineering to Arts

Indice

1	Introduzione	4
1.1	Obiettivi	4
1.2	Metodologia in breve	4
1.3	Requisiti	5
1.3.1	Requisiti funzionali	5
1.3.2	Requisiti non funzionali	5
1.4	Criticità	5
1.5	Specifiche del progetto	5
2	Dataset e librerie	6
2.1	Dataset	6
2.1.1	PhonItaliaR	6
2.1.2	Q2Stress	6
2.1.3	NRC	7
2.1.4	lakh-midi	7
2.2	Librerie	7
2.2.1	Pythorch	7
2.2.2	Pyphen	7
2.2.3	re	8
2.2.4	UnicodeData	8
2.2.5	Dataclasses	8
2.2.6	mido	8
3	Dal testo allo schema ritmico	9
3.1	Strumenti per l'analisi del testo	9
3.2	Sillabazione	10

3.2.1	Motore euristico primario	10
3.2.2	Processo di riconciliazione	11
3.2.3	Meccanismo di fallback e architettura completa	11
3.3	Accento tonico	12
3.3.1	Random Forest	13
3.3.2	Preparazione del dataset di addestramento	14
3.3.3	Addestramento del modello	14
3.3.4	Inferenza e validazione strutturale	15
3.4	Individuazione delle pause ritmiche	15
3.4.1	Tokenizzazione e analisi della punteggiatura	16
3.4.2	Gestione transizioni di fine verso	16
3.5	Sintesi MIDI dello schema ritmico	17
4	Analisi emotiva del testo	19
4.1	Analisi emotiva del testo	19
5	Addestramento del modello	20
5.1	Transformer	20
5.1.1	Meccanismo dell'auto-attenzione	20
5.1.2	Query, Key e Value	22
5.2	Training del modello	23
5.3	Selezione Automatica degli Strumenti	26
6	Rendering audio	27
6.1	Protocollo MIDI	27
6.2	Costruzione del file	28
6.3	Synth	29
6.3.1	Sintetizzatore matematico interno	29
6.3.2	FluidSynth	29
7	Conclusioni	32
7.1	Valutazione finale	32
7.2	Sviluppi futuri	32
8	"Da rivedere"	33

8.1	Approccio generale	33
8.2	Analisi Prosodica	33
8.3	Librerie usate	33
8.3.1	re	33
8.3.2	UnicodeData	34
8.3.3	Pyphen	34
8.4	Algoritmo	34
8.4.1	Analisi prosodica	34
8.4.2	Produzione dello schema ritmico	36
8.4.3	Scelta degli strumenti	36
8.4.4	Produzione della melodia	36
8.5	Analisi emotiva	36
8.6	Librerie	36
8.6.1	re	36
8.7	Algoritmo	37
A	Bibliografia e sitografia	38

Capitolo 1

Introduzione

Il progetto nasce dall'idea di poter associare una melodia ad un testo poetico. Molto spesso quando si legge un testo poetico, la mente del lettore potrebbe associare inconsciamente una melodia al testo stesso. Difatti la lingua italiana è essa stessa ritmata e bella da ascoltare, non c'è quindi da stupirsi se la sola lettura di una poesia possa far suonare nella propria mente una determinata colonna sonora.

Questo progetto mira a creare una pipeline che possa rendere tutto questo realtà trasformando un testo poetico italiano in una melodia coerente con il testo stesso.

Questa introduzione mira a fornire a chi legge una panoramica degli obiettivi e delle metodologie utilizzate in questo progetto.

1.1 Obiettivi

Il progetto ha come obiettivo principale la definizione di una pipeline coerente per trasformare un input testuale in una rappresentazione musicale espressiva e controllabile. Nello specifico il progetto mira a raggiungere i seguenti obiettivi:

Il progetto mira al raggiungimento dei seguenti obiettivi:

1. Prendere in input un testo ed estrapolare secondo le regole della lingua italiana le varie tipologie di accenti.
2. Produrre uno schema ritmico musicale coerente con la distribuzione degli accenti ritmici del testo.
3. Generare una melodia coerente con il ritmo ottenuto precedentemente lasciando libertà di personalizzazione all'utente.

1.2 Metodologia in breve

Al fine di raggiungere gli obiettivi precedentemente presentati sarà seguita la seguente pipeline:

1. **Analisi del testo:** L'analisi del testo sarà svolta parallelamente dal punto di vista semantico e da quello prosodico (degli accenti).
L'analisi semantica sarà effettuata per "assegnare" la musica corretta in base al significato e al contesto emotivo del testo.
L'analisi prosodica sarà usata con il fine di trovare gli accenti per la produzione dello schema ritmico.
2. **Produzione dello schema ritmico:** Traduzione dello studio prosodico in un ritmo musicale, in modo da ottenere una ritmica coerente con il testo.
3. **Addestramento del modello:** Per la generazione della melodia, in questa fase si addestrerà un modello Transformer su un dataset di brani musicali.
4. **Generazione musicale:** Sarà generato un file MIDI coerente con il ritmo e con la semantica del testo, per fare ciò si farà uso il modello Transformer precedentemente allenato.
5. **Sintesi del brano:** Il file precedentemente ottenuto darà sintetizzato (quindi trasformato in un file WAVE) utilizzando gli strumenti necessari.

1.3 Requisiti

In questa sezione saranno presentati i requisiti del progetto che saranno divisi in requisiti funzionali e requisiti non funzionali. I requisiti funzionali sono un'insieme di requisiti necessari al fine del funzionamento del progetto, se anche uno di loro non viene rispettato allora il progetto è irrealizzabile.

I requisiti non funzionali sono invece dei cosiddetti "nice to have" ovvero delle aggiunte che seppur non necessarie possono migliorare di molto sia la qualità nel lavoro che i risultati del progetto.

1.3.1 Requisiti funzionali

1.3.2 Requisiti non funzionali

1.4 Criticità

1.5 Specifiche del progetto

Lo strumento principale usato per la realizzazione di questo progetto sarà Python, in quanto dotato di numerose librerie che saranno presentate in un capitolo dedicato, estremamente utili allo svolgimento del progetto. Inoltre per l'addestramento dei modelli si farà uso di dataset reperibili online ricchi di informazioni utili.

Capitolo 2

Dataset e librerie

2.1 Dataset

Nell'ambito del machine learning e del deep learning per l'apprendimento di un modello è necessario fare uso di dati di training e di test che saranno prelevati da vari dataset reperibili online. Per questo progetto sono stati utilizzati i seguenti dataset:

2.1.1 PhonItaliaR

PhonItaliaR fornisce rappresentazioni fonologiche per 120.000 forme di parole italiane. Ognuno di questi include anche confini sillabici, marcature di accento e una gamma completa di statistiche lessicali. Utilizzando dati derivati da questo lessico, è anche possibile generare un insieme di database derivati e fornito stime dell'uso della frequenza posizionale per fonemi, sillabe, inizi e coda sillabiche italiane, nonché bigrammi di caratteri e fonemi.

Fonte: <https://link.springer.com/article/10.3758/s13428-013-0400-8>

In questo progetto il dataset PhonItaliaR è estremamente importante.

Sarà usato per individuare su quale sillaba cade l'accento di una parola. È annotato da linguisti su 120.000 forme, non dedotto da regole, è quindi la fonte più affidabile.

Inoltre verrà fatto uso di PhonItaliaR anche per il conteggio delle sillabe, l'integrazione più recente fornisce il numero "vero" di sillabe (SumSylls), usato per correggere la sillabazione quando l'euristica interna sbaglia (tipicamente sui casi di iato: "poesia", "farmacia").

2.1.2 Q2Stress

Q2Stress è un database per l'italiano che contiene informazioni su molteplici segnali che i lettori possono utilizzare per assegnare l'accento. In particolare, il database offre misure di frequenza di tipo e token dei pattern di accento in funzione del numero di sillabe, della categoria grammaticale, dell'inizio delle parole, delle terminazioni delle parole e delle strutture consonanti-vocali. Sono disponibili dati sia per adulti, sia per i bambini.

Q2Stress può aiutare i ricercatori a rispondere a domande empiriche e teoriche sull'assegnazione dell'accento e sulla relazione tra ortografia e fonologia nella lettura. Q2Stress è progettato come una risorsa facile da usare, poiché include script che permettono ai ricercatori di esplorare e selezionare i propri stimoli secondo diversi criteri, oltre a tabelle riassuntive per l'analisi complessiva dei dati. Fonte: <https://www.istc.cnr.it/en/content/q2stress>.

In questo progetto il dataset Q2Strss sarà usato in sinergia con PhonItaliaR per l'analisi fonetica. In particolare Q2Stress in questo progetto sarà un "ripiego", per quando PhonItalia non ha la parola (neologismi, forme rare, parole poetiche/arcaiche). Invece di ricadere subito sull'euristica grezza ("parola piana di default"). Q2Stress guarda la desinenza delle ultime 3 lettere e restituisce la posizione dell'accento più probabile secondo le statistiche reali sull'italiano (es. l'80% delle parole che finiscono in "-ino" sono piane).

2.1.3 NRC

2.1.4 lakh-midi

Il dataset MIDI di Lakh è una raccolta di 176.581 file MIDI unici, di cui 45.129 sono stati abbinati e allineati alle voci del Million Song Dataset. Il suo obiettivo è facilitare il recupero su larga scala delle informazioni musicali, sia simbolico (usando solo i file MIDI) sia basato su contenuti audio (utilizzando informazioni estratte dai file MIDI come annotazioni per i file audio abbinati).

Fonte: <https://colinraffel.com/projects/lmd/>

Nel contesto di questo progetto il dataset lakh-midi si occupa della parte della generazione del file MIDI.

2.2 Librerie

2.2.1 Pytorch

PyTorch è una libreria tensoriale ottimizzata per il deep learning utilizzando GPU e CPU. Fonte: <https://docs.pytorch.org/docs/main/generated/torch.sort.html> PyTorch viene utilizzato per caricare il modello pre-addestrato, gestire la memoria e campionare le nuove note musicali in fase di inferenza. Nello specifico è possibile vedere come `torch.nn` sia usato per la costruzione dell'architettura neurale. Si farà inoltre uso di `torch.softmax` / `F.softmax` che permettono l'uso della funzione di attivazione omonima che converte le uscite grezze dal modello in una distribuzione di probabilità valida compresa tra 0 e 1.

2.2.2 Pyphen

Pyphen è una libreria Python dedicata alla sillabazione automatica e alla ricerca dei punti di divisione delle parole, basata su dizionari di sillabazione compatibili con il formato utilizzato da

Hunspell. La libreria mette inoltre a disposizione dizionari per numerose lingue, tra cui l'italiano. (da chatgpt)

2.2.3 re

Questa libreria offre delle operazioni svolgibili sulle espressioni regolari.

Un'espressione regolare specifica un set di stringhe che combacia con essa. Le funzioni di questa libreria permettono di verificare se una particolare stringa combacia con una particolare espressione regolare. Le espressioni regolari possono essere concatenate per formare nuove espressioni regolari; se A e B sono entrambe espressioni regolari, allora AB è anch'essa un'espressione regolare. In generale, se una stringa p corrisponde ad A e un'altra stringa q corrisponde a B, la stringa pq corrisponderà ad AB. Questo vale a meno che A o B contengano operazioni a bassa priorità; condizioni ai confini tra A e B; o riferimenti a gruppi numerati. Fonte: <https://docs.python.org/3/library/re.html>

Nel codice implementato la libreria re serve per usare la funzione re.sub che rimuove tutti i caratteri di punteggiatura da una parola mantenendo solo le lettere e i caratteri accentati.

2.2.4 UnicodeData

La libreria dà accesso all'Unicode Character Database (UCD) che definisce le proprietà dei caratteri di tutti i caratteri Unicode.

Fonte: <https://docs.python.org/3/library/unicodedata.html>

Viene importata per gestire correttamente la codifica dei caratteri speciali e accentati. In particolare viene usato il metodo lookup che cerca un carattere dal nome. Se un carattere con tale nome viene trovato ritorna il carattere corrispondente, altrimenti viene lanciato un Keyerror.

2.2.5 Dataclasses

2.2.6 mido

Capitolo 3

Dal testo allo schema ritmico

(**Analisi prosodica del testo**) La prosodia è la parte della linguistica e della grammatica che studia gli elementi sonori del discorso, come l'intonazione, il ritmo, l'accento, la durata e le pause. In questa fase iniziale della pipeline avviene l'analisi prosodica di un testo di input, con l'obiettivo di produrre una rappresentazione ritmico-temporale che sarà la base di partenza per la fase successiva di generazione musicale. Per questo progetto la lingua di riferimento è l'italiano, con le sue regole grammaticali, fonetiche e sugli accenti.

3.1 Strumenti per l'analisi del testo

Prima di procedere con l'analisi prosodica vera e propria, è fondamentale definire formalmente le regole fonetiche e ortografiche della lingua italiana nell'ambiente computazionale. Affinché il sistema possa interpretare correttamente il ritmo di un verso, deve prima essere istruito a riconoscere le unità cardine che compongono le parole.

Una prima distinzione avviene nella classificazione dei grafemi vocalici e di tutte le possibili varianti accentate; il sistema distingue tra *vocali forti* (quali la "a", "e", "o", includendo le loro forme accentate e quelle di "i" e "u") e *vocali deboli* (come "i" e "u"). Questa suddivisione è importante per la fase successiva di sillabazione perché permette la discriminazione di possibili *dittonghi* (vocali deboli adiacenti che costituiscono un'unica sillaba) e *iati* (vocali forti adiacenti appartenenti a due sillabe diverse).

Un'altra classificazione è quella dei *digrammi*, ossia sequenze di due lettere che rappresentano un unico suono (come "ch", "gh", "gn" e "gl"). L'identificazione a priori di queste stringhe è essenziale per evitare che l'algoritmo di sillabazione separi le lettere che costituiscono un suono indivisibile. Un'attenzione particolare è riservata al digramma "sc" che richiede regole contestuali: è trattato come tale solo quando è seguito dalle vocali "e" o "i" (es. scena), mentre viene separato in altri contesti (es. scala).

Per facilitare l'individuazione deterministica dell'accento tonico (**descritta nel sotto-paragrafo X**), è stata implementata una mappa di conversione che associa ogni lettera con accento grafico alla sua corrispondente vocale non accentata. Quando una parola presenta un accento grafico, questo coincide sempre con l'accento tonico principale di quella parola.

Infine, il sistema si compone anche di uno strumento di normalizzazione del testo. Tramite l'uso di espressioni regolari (regex, **riferimento alla libreria...**) il testo in ingresso viene "ripulito" per facilitarne le successive analisi: ogni parola è convertita in minuscolo, vengono rimossi gli spazi bianchi e gli apostrofi, tutti i caratteri non alfabetici sono filtrati. Questo processo di standardizzazione dell'input riduce il rumore nei dati, preparando una stringa uniforme, pronta per essere sottoposta alla successiva fase di suddivisione sillabica.

3.2 Sillabazione

La scomposizione di una parola in sillabe rappresenta il fondamento su cui si costruisce l'intera architettura ritmica del sistema. Per la successiva generazione musicale, è fondamentale che la sillabazione rispecchi l'effettiva pronuncia e la scansione metrica del verso poetico. Il sistema implementa un approccio ibrido a cascata, basato su un motore euristico principale supportato da meccanismi di riconciliazione su base dizionariale e, solo in ultima istanza, da strumenti tipografici.

3.2.1 Motore euristico primario

Il primo livello di analisi è affidato alla funzione di classificazione basata su regole linguistiche. L'euristica responsabile della sillabazione riceve in input una parola e restituisce una lista di sillabe. L'analisi avviene scorrendo sequenzialmente la stringa per individuare i "nuclei vocalici" e i successivi gruppi consonantici. Dall'analisi dei gruppi consonantici è possibile stabilire il punto esatto in cui termina la sillaba corrente:

- Se tra un nucleo vocalico e il successivo non sono presenti consonanti, allora il nucleo è composto da due vocali forti, dunque il sistema rileva uno iato e separa le due vocali (es. "pa-e" di paese).
- Se il gruppo consonantico è composto da un'unica consonante, allora la sillaba corrente termina subito prima di essa (es. "ca-sa").
- Nel caso in cui il gruppo consonantico comprenda più elementi, si prendono in considerazione le seguenti regole relativi alle prime due consonanti del gruppo:
 - Se le due consonanti costituiscono un digramma, che sia riconosciuto per mapping con una lista interna del sistema che corrispondano a "sc" seguito da "i" o "e", allora la sillaba termina prima del digramma (es. "ba-gno" oppure "pe-sce").
 - Se le due consonanti sono uguali (doppie), esse vengono separate in due sillabe diverse (es. "gat-to").
 - Se sono presenti esattamente due consonanti e la seconda è una "l" oppure una "r", allora la sillaba termina prima del gruppo consonantico (es. "li-bro").
 - In tutti gli altri casi, la divisione sillabica avviene in modo da isolare l'ultima consonante del gruppo, che andrà a costituire l'inizio della sillaba successiva.

L'algoritmo itera questo processo fino alla scomposizione in sillabe completa della parola.

3.2.2 Processo di riconciliazione

Poiché la lingua italiana presenta numerose eccezioni storiche e fonetiche difficilmente individuabili con l'utilizzo di sole euristiche, viene introdotto un livello di validazione incrociata. Non sempre vocali miste (forti e deboli) adiacenti si comportano come dittonghi, bensì spesso sono considerate iati a causa dell'etimologia della parola. Questo può portare il precedente metodo di suddivisione sillabica a sottostimare il numero effettivo di sillabe, compromettendo l'analisi prosodica del testo.

Il risultato calcolato dall'euristica viene confrontato con le annotazioni di un dizionario fonetico di riferimento: PhonItalia (**descritto nel capitolo X**). Se il numero di sillabe calcolato differisce dal conteggio reale presente nel dataset, si avvia una procedura di riconciliazione dinamica. Le sillabe della parola ricevuta in input vengono poste in ordine decrescente rispetto al numero di vocali che ciascuna contiene. Viene selezionata la sillaba con un maggior numero di vocali perché candidata principale a contenere uno iato non individuato. In seguito, avviene un tentativo di scissione nel punto che separa le due vocali adiacenti. Se la separazione è attuabile, la vecchia struttura viene sostituita con le due nuove unità sillabiche. A questo punto, il sistema verifica se il numero totale di sillabe ha raggiunto il valore target riportato nel dizionario: in caso affermativo, la procedura termina con successo; altrimenti, il processo di ordinamento e scissione viene iterato. Le iterazioni sono limitate da un numero di guardia (in questo caso 5), oltre il quale l'algoritmo si interrompe con la restituzione del risultato parziale calcolato.

3.2.3 Meccanismo di fallback e architettura completa

Come meccanismo di sicurezza finale, o fallback, il sistema prevede l'utilizzo della libreria tipografica Pyphen nei casi in cui la sillabazione euristica e il successivo meccanismo di riconciliazione non producano un risultato coerente con quello riportato nel dizionario. Questa libreria nasce come strumento di *hyphenation*, ossia per determinare possibili punti di divisione tipografica nelle parole, non con l'obiettivo di attuare un'analisi prosodica. Nel contesto del sistema, infatti, la sillabazione non rappresenta un fine autonomo, ma costituisce una fase intermedia dell'analisi prosodica, dalla quale vengono ricavate informazioni utili nella costruzione dello schema ritmico. Basandosi su pattern predefiniti e regole deterministiche associate ai dizionari linguistici, Pyphen può portare a suddivisioni non conformi alla reale scansione metrica, pertanto non è opportuno affidarsi ad essa come motore primario di sillabazione. Tuttavia, la sua inclusione risulta essenziale per garantire la robustezza dell'intera pipeline di suddivisione sillabica:

1. In prima istanza, la stringa viene sottoposta al motore euristico basato su regole fonetiche.
2. Successivamente l'output viene validato con il dizionario PhonItalia. Qualora emerga una discrepanza nel computo sillabico, viene innescato il meccanismo di riconciliazione dinamica.

3. In caso di fallimento del tentativo di riconciliazione, il sistema interroga Pyphen e adotta l'output ottenuto solo nell'eventualità in cui coincida con il conteggio atteso; altrimenti utilizza il risultato parziale computato con la riconciliazione.
4. In caso di fallimento totale dell'euristica e in assenza di riscontri nel dizionario, Pyphen interviene come ultimo tentativo. Se anche in tal caso non viene prodotto un risultato corretto, l'algoritmo restituisce l'intera parola come macro-sillaba, evitando così interruzioni o errori irreversibili.

Conclusa questa architettura a cascata, il sistema ottiene per ogni parola una lista strutturata e metricamente coerente delle sillabe che la compongono. Tuttavia, per poter tradurre queste unità testuali in un reale schema ritmico-musicale, la separazione sillabica non è sufficiente. Il passaggio immediatamente successivo, fondamentale per l'analisi prosodica, consiste nell'individuazione della sillaba tonica, ossia contenente l'accento tonico primario.

3.3 Accento tonico

Estremamente importante per la scrittura dello schema ritmico è la disposizione degli accenti tonici delle parole, essi diventeranno gli "hit" che guideranno la ritmica (?). La loro individuazione avviene mediante una catena a 5 livelli, il cui livello più sofisticato riporta l'utilizzo di un modello di Machine Learning spiegato in una sotto-sezione apposita.

A partire dalla parola e dalla lista delle sue sillabe, la ricerca si articola nelle seguenti fasi:

1. Accento grafico esplicito: è il primo livello di verifica e ha la priorità assoluta in quanto ottimizza notevolmente l'intero meccanismo. In ortografia italiana, quando una parola presenta un accento grafico, esso coincide con l'accento tonico. Di conseguenza, il sistema cerca la presenza di lettere accentate nella parola, utilizzando la mappa di conversione definita tra gli strumenti per l'analisi del testo. In caso di riscontro positivo, restituisce la sillaba accentata.
2. Lookup nel dizionario: in assenza di accenti grafici espliciti, il sistema effettua la ricerca nel dizionario fonetico PhonItalia. Se la parola ha un riscontro nel database, viene restituito l'indice della sillaba tonica accentata.
3. Modello di Machine Learning: qualora la parola risulti assente nel dizionario, viene analizzata da un modello di Machine Learning supervisionato basato sull'algoritmo Random Forest. Analizzando le caratteristiche morfologiche della stringa, esso è in grado di predire la classe di accentazione.
4. Predizione statistica per desinenza: in caso di assenza del modello di ML o di restituzione di esiti non validi, entra in gioco il modulo probabilistico Q2Stress. Questo livello calcola la probabilità di posizione dell'accento basandosi unicamente sulle ultime tre lettere della parola, sfruttando le distribuzioni statistiche tipiche della lingua italiana.

5. Euristica di default: come livello di sicurezza finale, nel caso di fallimento dei metodi precedenti, l'algoritmo assegna l'accento tonico di default alla penultima sillaba, individuando una parola piana. Nel caso limite in cui la parola sia monosillabica, l'accento ricade inevitabilmente sull'ultima sillaba.

Attraverso questo meccanismo a cascata, l'algoritmo assicura l'individuazione dell'accento tonico per qualunque parola passata in input.

3.3.1 Random Forest

Quando la parola non è rintracciabile nel dizionario di riferimento, il sistema delega la ricerca della sillaba tonica a un modello di Machine Learning basato sui Random Forest. Il Random forest è un algoritmo di apprendimento supervisionato di tipo ensemble che, durante la fase di addestramento, costruisce numerosi alberi decisionali indipendenti e poi ne confronta i risultati. Ogni decision tree considera solo un sottoinsieme randomico di caratteristiche, in modo da esplorare complessivamente lo spazio delle feature mantenendo unici i singoli percorsi, mitigando il rischio di overfitting. Al termine del processo, la previsione finale viene determinata per maggioranza, selezionando la classe di appartenenza stimata più dalla maggioranza degli alberi. L'implementazione è stata effettuata tramite la libreria Python di Machine Learning Scikit-Learn. L'obiettivo di questo classificatore è predire la posizione dell'accento tonico mappando la parola nelle seguenti classi di appartenenza:

- Ultima sillaba (parole tronche)
- Penultima sillaba (parole piane)
- Terzultima sillaba (parole sdrucciole)
- Quartultima sillaba (parole bisdruciole)

Per consentire al modello di generalizzare le regole di accentazione, ogni stringa viene trasformata in un vettore di 24 feature descrittive, riportate di seguito:

- Caratteristiche generali della parola:
 - Lunghezza complessiva della parola calcolata come numero di lettere (`word_len`).
 - Conteggio di vocali (`vowel_count`) e consonanti (`consonant_count`).
 - Rapporto percentuale tra vocali e lunghezza totale della parola (`vowel_ratio`).
 - Numero delle sillabe della parola, precedentemente ottenuto nella fase di sillabazione (`syll_count`).
 - Indicatori posizionali di chiusura rappresentati da flag binari che segnalano se la parola termina con una vocale (`ends_with_vowel`) o una consonante (`ends_with_consonant`). Possono costituire un forte indizio di parole tronche.

- Sequenze vocaliche: flag booleani che verificano la presenza interna di specifiche combinazioni di vocali forti e deboli. Influenzano notevolmente la struttura sillabica e la posizione dell'accento tonico. Sono rappresentate dalla feature "contains_x" con x che racchiude tutte le possibili combinazioni: "ia", "ie", "io", "iu", "ua", "ue", "ui", "uo".
- Sequenze consonantiche: intercettano pattern consonantico-vocalici ricorrenti con la lettera "t". Rappresentate con "contains_x" con x indicante: "ta", "te", "ti", "to", "tu".
- Desinenze: stringhe estratte dalla porzione finale della parola, corrispondenti all'ultima lettera (last1), alle ultime due (last2), tre (last3) e quattro (last4). Sono le feature categoriche più importanti perché codificano direttamente le desinenze morfologiche (es. -are, -issimo). Trattate con la codifica One-Hot Encoding di skit-learn, consentono di associare regolarità statistiche fisse alle relative classi di accento.

3.3.2 Preparazione del dataset di addestramento

Per addestrare il modello il sistema unisce due dataset distinti: PhonItalia e Q2Stress. Usare più di un dataset permette di massimizzare la copertura lessicale per garantire una maggiore robustezza statistica. L'elaborazione procede nel seguente modo:

- Lessico di Phonitalia: i dati estratti dal dataset contengono la forma della parola, il conteggio delle sillabe e la posizione dell'accento calcolata a partire dalla fine della stringa (dove 1 corrisponde all'ultima sillaba). Poiché il classificatore richiede una codifica 0-based (a partire da 0), la funzione converte l'etichetta originaria in un target tra 0 e 3. Ciascuna voce viene poi convertita nel vettore di 24 feature.
- Dataset word-level Q2Stress: per questo dizionario la codifica del pattern di accento è già formattata in base 0, pertanto non necessita di conversioni. Inoltre, quando il dataset fornisce esplicitamente il conteggio delle sillabe, questo parametro ha la precedenza assoluta rispetto all'euristica di sillabazione.

Al termine di queste elaborazioni, le due fonti vengono fuse in un unico dataframe complessivo ed epurate da eventuali record duplicati, fornendo una base di apprendimento omogenea, densa e priva di ridondanze.

3.3.3 Addestramento del modello

Una volta strutturato il dataset, il modello viene addestrato seguendo varie fasi, incapsulate in un oggetto `Pipeline` della libreria `scikit-learn`. La pipeline si articola in:

1. **Preprocessing dei dati:** il vettore delle 24 feature presenta una natura eterogenea. Tramite `ColumnTransformer` vengono separati i flussi di informazione: le feature numeriche vengono normalizzate utilizzando uno `StandardScaler`, mentre le feature categoriche vengono trasformate in vettori binari tramite `OneHotEncoder`. Quest'ultimo codificatore permette di ignorare le categorie sconosciute, assicurando che il sistema non vada in errore.

2. **Bilanciamento dei pesi di una classe:** il lessico italiano presenta una netta predominanza statistica delle parole piane. Per evitare che il classificatore sviluppi un bias predittivo orientato esclusivamente verso la classe dominante, viene implementato un calcolo personalizzato dei pesi. Analizzando le frequenze del dataset, il sistema calcola dei pesi compensativi e li attenua tramite un parametro di smorzamento, restituendo la corretta importanza anche alle classi minoritarie.
3. **Configurazione del classificatore:** l'algoritmo predittivo finale è un `RandomForestClassifier` impostato per generare 400 alberi decisionali. Per attenuare il rischio di overfitting, viene imposto un vincolo strutturale che richiede un minimo di due campioni per la creazione di un nodo foglia.

L'incapsulamento di questi tre passaggi all'interno di un'unica struttura dati garantisce che il modello addestrato possa processare nuove parole in modo coerente e sicuro.

3.3.4 Inferenza e validazione strutturale

Completata la fase di addestramento, il modello è pronto ad elaborare nuove parole fuori dal vocabolario. Tuttavia, l'algoritmo non adotta la predizione assoluta restituita dal classificatore, ma implementa un meccanismo di validazione strutturale. Il processo di inferenza si articola nei seguenti passaggi:

1. Definizione dei limiti strutturali: sulla base del conteggio sillabico della parola, viene calcolata la classe metrica massima ammissibile.
2. Valutazione probabilistica: invece di ottenere una predizione netta e univoca, vengono estratte le probabilità stimate per ciascuna delle quattro classi possibili.
3. Filtraggio e selezione: l'algoritmo, seguendo i limiti strutturali, scarta tutte le probabilità delle classi non ammissibili per quella parola. Tra le classi candidate, viene selezionata quella con probabilità più alta.

Questo approccio ibrido, che unisce la potenza statistica del Machine Learning ai vincoli logico-deduttivi della metrica, assicura la generazione di output coerenti e previene errori strutturali irreversibili nella successiva stesura dello schema ritmico.

Inoltre, per garantire prestazioni ottimali ed evitare di ripetere il gravoso processo di training per ogni parola analizzata, l'intera pipeline addestrata viene memorizzata in un meccanismo di caching interno a livello di sessione. In questo modo, l'addestramento avviene un'unica volta all'avvio, e la memoria si azzer automaticamente al termine dell'esecuzione.

3.4 Individuazione delle pause ritmiche

Dopo aver trovato le sillabe di maggior intensità sonora delle parole, il passo successivo per costruire un efficace schema ritmico ripone la sua attenzione nell'analisi dei silenzi. La punteggiatura e il modo di trascrivere una poesia, sono elementi rilevanti per l'individuazione delle

pause ritmiche. Tali pause sono indispensabili per avvicinarsi ad un'analisi prosodica di un testo esposto vocalmente, in quanto esse simulano l'andamento che la voce è portata a seguire.

Per lo studio dei silenzi, è necessaria una definizione di un meccanismo di tokenizzazione dei caratteri testuali per riconoscere la punteggiatura, e di interpretare pattern impliciti nascosti nei versi come gli enjambement.

3.4.1 Tokenizzazione e analisi della punteggiatura

La scomposizione del testo poetico è basata sull'uso della libreria Python Regex (**riferimento alla libreria X**) che permette l'implementazione di espressioni regolari ad hoc. Si è scelto di utilizzare questa libreria leggera in quanto ottimizza notevolmente la computazione, ed evita il sovraccarico computazionale che avrebbero arrecato librerie di NLP complesse come spaCy. Il testo di input viene analizzato verso per verso, e per ciascuno, viene restituita una sequenza strutturata di dizionari contenenti una parola e l'eventuale segno di punteggiatura immediatamente successivo.

Una volta isolata la punteggiatura del testo, viene mappata su specifiche durate musicali, misurate in "impulsi" (dove in notazione MIDI corrisponde a una croma o a 0.5 quarterLength). Il sistema distingue tre categorie di pause infra-verso:

- Pause brevi: dedicate a virgole, punti e virgola o due punti, a cui viene assegnata una durata di 1 impulso.
- Pause lunghe: riguardanti punti, punti esclamativi o punti interrogativi, che impongono un arresto maggiore pari ai 3 impulsi.
- Pause sospensive: attivate dai punti di sospensione che dilatano ulteriormente il silenzio estendendolo a 4 impulsi.

In assenza di punteggiatura, le parole vengono concatenate musicalmente senza pause aggiuntive. Questa mappatura diretta tra tipografia e valori ritmici permette al sistema di emulare la metrica respiratoria della recitazione.

3.4.2 Gestione transizioni di fine verso

La sola analisi della punteggiatura intra-verso risulta insufficiente a modellare la complessità ritmica della poesia, poiché la transizione tra un verso e il successivo impone intrinsecamente una pausa strutturale. Tuttavia, tale arresto deve essere ricalibrato in presenza di un enjambement, ovvero quando la struttura sintattica si protrae da un verso all'altro.

Per identificare automaticamente questi contesti, il sistema si avvale di un metodo euristico comprendente una lista mirata di elementi grammaticali a forte propensione sintattica trasversale. L'insieme comprende:

- Preposizioni e articoli: sia semplici che articolati (es. *di, a, da, nello, del*), la cui funzione di reggenza nominale impedisce per definizione un'interruzione di respiro.

- Congiunzioni e particelle subordinanti: elementi connettivi che legano indissolubilmente sintagmi o introducono proposizioni dipendenti (es. *che, e, ma, o*).
- Pronomi atoni e clitici: particelle proclitiche o enclitiche prive di autonomia accentuativa (es. *mi, ti, si, ne*).

Quando l'ultima parola di un verso appartiene a questo insieme, l'algoritmo rileva un potenziale enjambement. In risposta, il sistema attenua la durata della pausa a fine verso, impedendo che venga inserito un silenzio innaturale laddove la sintassi impone continuità ritmica.

Inoltre, per evitare artefatti acustici dovuti alla sovrapposizione temporale, viene attuato un controllo di mutua esclusività: se l'ultimo token di un verso presenta già un segno di punteggiatura, la pausa strutturale di fine verso viene annullata. Questo meccanismo impedisce la somma di silenzi consecutivi, preservando la coerenza ritmica.

3.5 Sintesi MIDI dello schema ritmico

La fase conclusiva di tutta questa prima fase di analisi dell'input, traduce la rappresentazione prodotta dall'analisi prosodica del testo in un file MIDI standard (.mid), avvalendosi della libreria di Python music21 (**riferimenti alla libreria e al MIDI**). Il processo si articola in due momenti logici principali: la linearizzazione della struttura in una sequenza ordinata di eventi discreti e la loro successiva conversione in oggetti musicali, come note e pause, organizzate in uno stream temporizzato.

Nel dettaglio:

- Linearizzazione e allineamento temporale: la struttura gerarchica dell'analisi prosodica, viene appiattita in un'unica sequenza monodimensionale di eventi, classificati come note o pause. Le pause vengono inserite nella sequenza subito dopo la sillaba a cui sono state associate, così da mantenere la corrispondenza tra accenti linguistici e silenzi strutturali.
- Sistema di durate a impulsi: ad ogni evento una durata espressa in un'unità ritmica astratta, l'impulso: uno per le sillabe atone, due per le toniche, e per le pause coerentemente all'analisi discussa sopra. In fase di esportazione, l'impulso viene convertito nell'unità temporale nativa di music21 (**riferimenti...**), la `quarterLength`, tramite un fattore di scala fisso, dove un impulso equivale a una croma (0.5 `quarterLength`).
- Mappatura acustica e profili dinamici: per rendere udibile la gerarchia accentuativa, alle sillabe toniche e atone vengono assegnate altezze fisse distinte (di default rispettivamente C4 e A3) e valori di `velocity` differenti (100 e 60 su una scala 0–127), replicando a livello sonoro il contrasto tra elementi forti e deboli individuato nell'analisi linguistica.
- Canale percussivo General MIDI: in alternativa alla riproduzione intonata, il sistema supporta una modalità di esportazione che instrada gli eventi sul canale 10, riservato dallo standard General MIDI ai suoni percussivi non intonati. Le sillabe toniche vengono mappate sulla nota della grancassa (**bass drum**, MIDI 36) e quelle atone sull'**hi-hat** chiuso (MIDI 42), generando un pattern puramente ritmico.

- Serializzazione e metadati strutturali: gli eventi convertiti vengono accumulati in un'unica traccia (**Part**), preceduta da un marcatore di tempo espresso in BPM. il file viene poi serializzato su disco nel formato MIDI standard. A fini di controllo, la durata complessiva della traccia in quarterLength viene convertita in secondi sulla base del tempo impostato, garantendo la coerenza temporale tra il testo analizzato e il brano generato.

Questo modulo rappresenta quindi il punto di chiusura della prima fase della pipeline su cui si fonda l'intero sistema: dal prodotto dell'analisi prosodica del testo si è generato uno scheletro ritmico deterministico. La base ritmica prodotta diventerà l'input per la successiva fase di generazione musicale intelligente.

Capitolo 4

Analisi emotiva del testo

4.1 Analisi emotiva del testo

Capitolo 5

Addestramento del modello

Una volta completata l'analisi prosodica ed emotiva si può passare alla parte di produzione musicale.

In breve: Per svolgere questo task si farà prima uso di un transformer che si occuperà della parte riguardante la generazione del modello con i vari token (non ancora musicale quindi), e successivamente gli intervalli "grezzi" precedentemente generati saranno agganciati alla scala musicale scelta in base a prosodia ed emozione. Così facendo saranno create la melodia e l'armonia.

5.1 Transformer

Il transformer è un modello di apprendimento profondo che adotta il meccanismo della auto-attenzione, pesando differientemente la significatività di ogni parte dei dati in ingresso.

Come le reti neurali ricorrenti (RNN), i trasformatori sono progettati per processare dati sequenziali come, ad esempio, testi in linguaggio naturale, con applicazioni alla loro generazione e traduzione automatica. Tuttavia, a differenza delle RNN, i trasformatori elaborano l'intero insieme di dati d'ingresso simultaneamente. Il cosiddetto meccanismo di attenzione fornisce il contesto per ogni posizione nella sequenza di ingresso. Per esempio, se i dati rappresentano una frase, il trasformatore non deve elaborare una parola alla volta: questo permette una maggiore parallelizzazione rispetto alle RNN e perciò porta a ridurre i tempi di addestramento.

Fonte: <https://it.wikipedia.org/wiki/Trasformatore>

5.1.1 Meccanismo dell'auto-attenzione

Nello specifico il meccanismo della auto-attenzione che consente a queste reti di analizzare e ponderare l'importanza di ogni parte di una sequenza di dati rispetto a tutte le altre parti della stessa sequenza. Questo è particolarmente utile per comprendere il contesto di una parola in una frase, poiché una parola può influenzare o essere influenzata da qualsiasi altra parola, indipendentemente dalla distanza nella sequenza. Ogni parola o simbolo nella sequenza viene

convertito in un vettore numerico (embedding) che rappresenta il suo significato in uno spazio multidimensionale.

Un blocco del Transformer è una combinazione di diversi strati, ciascuno dei quali ha specifiche funzioni. Un tipico blocco di Transformer include:

- Strato di attenzione multi-head dove avviene la self-attention.
- Strato di normalizzazione che applica una normalizzazione ai dati per mantenere la stabilità e l'efficienza del modello.
- Feed-forward network: Una rete neurale semplice che elabora i dati ulteriormente, permettendo al modello di apprendere trasformazioni più complesse.
- Un altro strato di normalizzazione che applica una normalizzazione ai dati per mantenere la stabilità e l'efficienza del modello.

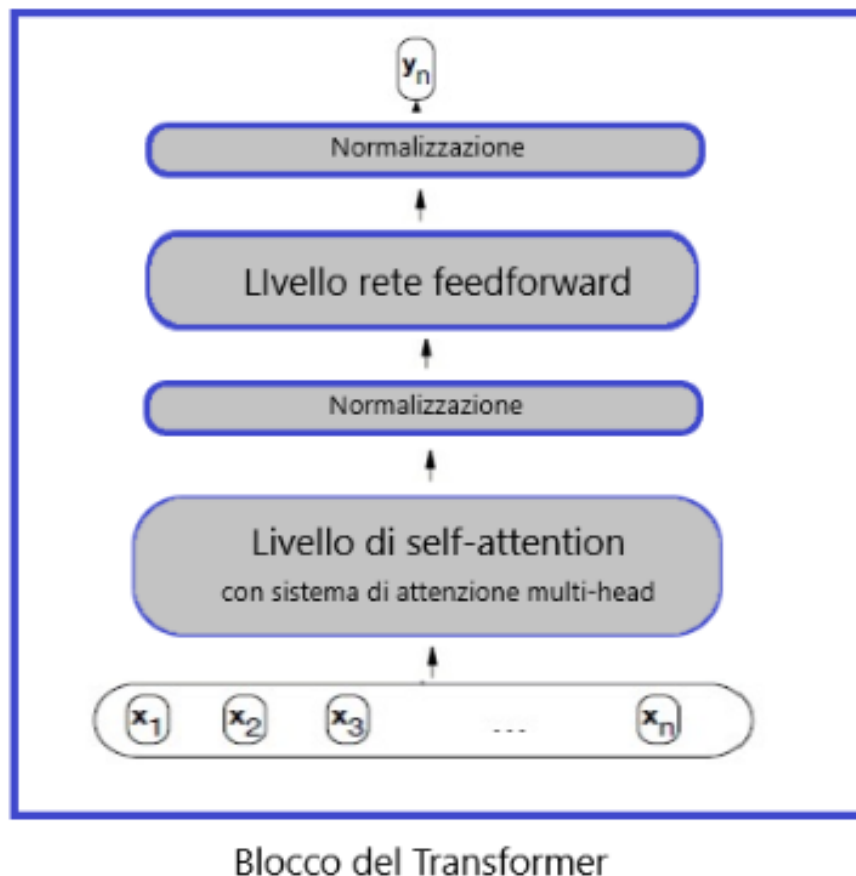


Figura 5.1: pipeline di un Transformer

La **Self-Attention** (auto-attenzione) è il nucleo matematico che permette di valutare la rilevanza contestuale di ogni token rispetto all'intera sequenza. Attraverso operazioni di vettorizzazione e trasformazione matriciale, essa converte le parole in rappresentazioni numeriche contestuali.

5.1.2 Query, Key e Value

Ogni elemento di un input x_i viene proiettato, tramite tre matrici di peso apprese W_Q, W_K, W_V , in tre vettori fondamentali:

$$q_i = W_Q x_i, \quad k_i = W_K x_i, \quad v_i = W_V x_i.$$

- **Query** (Q): rappresenta la domanda sulla rilevanza del token attualmente in esame rispetto al resto della sequenza. È essenzialmente il modo in cui il modello si chiede: Per ogni parola della sequenza viene generato un vettore di query, poi utilizzato per valutarne la relazione con le altre parole.
- **Keys** (K): rappresentano i punti di riferimento della sequenza. Ogni parola, inclusa quella attualmente focalizzata, possiede un vettore chiave corrispondente. Le chiavi fungono da termine di confronto per il vettore di query, aiutano cioè il modello a determinare quanto ciascuna parola della sequenza sia correlata alla parola attualmente focalizzata.
- **Values** (V): rappresentano il contenuto informativo pesato, ciò che il modello utilizza in definitiva per costruire la propria comprensione della sequenza. Ogni parola è associata a un vettore di valore, che ne contiene il significato contestuale. Una volta determinato — tramite il confronto tra query e chiavi — quanta attenzione dedicare a ciascuna parola, il modello utilizza i vettori di valore per costruire una rappresentazione ponderata del contesto.

La formulazione rigorosa dell'auto-attenzione, detta **Scaled Dot-Product Attention**, è data da:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V,$$

dove d_k è la dimensione dei vettori chiave (e query).

La scalatura è una necessità tecnica imprescindibile: in spazi ad alta dimensionalità, il prodotto scalare $QK^\top$ tende a crescere in modulo, portando gli argomenti della funzione Softmax in regioni in cui i gradienti sono estremamente piccoli (saturazione). Dividere per $\sqrt{d_k}$ stabilizza la varianza dei punteggi di attenzione e, di conseguenza, la backpropagation, prevenendo l'esplosione/evanescenza dei gradienti.

I punteggi scalati $\frac{QK^\top}{\sqrt{d_k}}$ passano attraverso la funzione Softmax, che li trasforma in una distribuzione di probabilità la cui somma è pari a 1:

$$\alpha_{ij} = \frac{\exp(q_i \cdot k_j / \sqrt{d_k})}{\sum_l \exp(q_i \cdot k_l / \sqrt{d_k})}.$$

I coefficienti α_{ij} determinano matematicamente quanta attenzione debba essere dedicata al token j rispetto al token i attualmente in esame. In questo senso, la Softmax agisce come un selettore probabilistico, identificando quale “slot” della rappresentazione (ad esempio una codifica posizionale) debba essere attivato in relazione al contesto.

I pesi ottenuti dalla Softmax vengono infine moltiplicati per i vettori dei valori V . Il risultato è una somma ponderata che genera un vettore di output arricchito:

$$z_i = \sum_j \alpha_{ij} v_j,$$

in cui ogni parola non è più un'entità isolata, ma incorpora informazioni provenienti dall'intera sequenza di input. L'uscita finale dell'attenzione si ottiene dunque dai pesi α_{ij} moltiplicati per V , ogni vettore di output z_i corrisponde a una parola (token) della sequenza, arricchita del contesto globale.

5.2 Training del modello

L'obiettivo dell'addestramento è insegnare alla rete la grammatica e la sintassi melodica generale (come le note si susseguono in modo musicale) e come piegare questa melodia in base al contesto emotivo (Valenza e Arousal).

L'addestramento funziona nelle seguenti fasi:

1. Invece di insegnare alla rete le note assolute (es. Do, Re, Mi), il modello impara gli intervalli melodici in semitoni tra una nota e la successiva (es. +2 per un salto di tono in avanti, -3 per una terza minore all'indietro). Un salto di +2 semitoni ha la stessa "funzione melodica" sia in Do maggiore che in Fa# maggiore. Questo riduce il vocabolario a soli 26 token (25 intervalli + 1 token di PAD dedicato). La mappatura dei token funziona attraverso la seguente formula:

$$\text{Token ID} = \text{Intervallo (in semitoni)} + 12$$

2. Per evitare l'overfitting sul corpus MIDI (composto da poche migliaia di file) e rendere il modello capace di generalizzare su qualsiasi nuova poesia, durante l'addestramento vengono applicate due tecniche fondamentali:

Per ogni sequenza di intervalli $[s_1, s_2, s_3, \dots]$, lo script genera la sua versione speculare $[-s_1, -s_2, -s_3, \dots]$. Questo raddoppia la dimensione del dataset e insegna alla rete che i movimenti melodici speculari sono egualmente validi, questa tecnica è anche detta Data Augmentation.

Nel 15% dei casi di addestramento, i valori di Valenza e Arousal passati alla rete vengono impostati forzatamente a zero $[0.0, 0.0]$. Si fa per Costringe il Transformer a imparare prima la struttura della sintassi musicale in modo indipendente ("incondizionato") e solo successivamente a correlarla alle emozioni, evitando che si affidi unicamente all'emozione ignorando la coerenza melodica, questa tecnica è nota come "Conditioning Dropout".

3. Il modello elabora il contesto attraverso due flussi che si uniscono:
 - Embedding Emotivo: La coppia di valori continui $(V, A) \in [-1, 1]$ passa attraverso uno strato denso (nn.Linear) che la proietta in un vettore a 128 dimensioni. Questo vettore viene concatenato all'inizio della sequenza come primo token (prefisso).

- Maschera causale: Durante l'addestramento, il Transformer vede l'intera sequenza contemporaneamente per calcolare il gradiente in parallelo. Per evitare che il modello "spii" le note future per predire quella attuale, si applica una maschera causale:

$$\text{Mask}_{ij} = \begin{cases} 0 & \text{se } i \geq j \quad (\text{può guardare solo al passato}) \\ -\infty & \text{se } i < j \quad (\text{bloccato}) \end{cases}$$

4. La rete viene addestrata con un approccio Self-Supervised (Autoregressivo): data una sequenza di intervalli fino al tempo t , la rete deve predire il token dell'intervallo al tempo $t + 1$.
5. L'input è la sequenza dal token 0 a $N - 1$, mentre il target da confrontare è la sequenza traslata da 1 a N . Misura la distanza tra le probabilità calcolate dalla Softmax della rete e l'effettivo token di destinazione:

$$\mathcal{L} = - \sum_{c=0}^{25} y_c \log(\hat{y}_c)$$

Per l'ottimizzazione viene usato l'ottimizzatore AdamW con Weight Decay (10^{-2}):

L'ottimizzatore Adam, acronimo di Adaptive Moment Estimation, è un sofisticato algoritmo di ottimizzazione ampiamente utilizzato per addestrare modelli di deep learning, calcolando learning rates adattivi individuali per diversi parametri dalle stime del primo e del secondo momento dei gradienti.

L'ottimizzatore AdamW è una variante dell'algoritmo Adam che corregge il modo in cui viene gestito il decadimento dei pesi (weight decay), migliorando la capacità di generalizzazione dei modelli di deep learning.

Penalizza quindi i pesi sinaptici che crescono troppo, impedendo alla rete di fossilizzarsi e il Dropout (0.2) che disattiva casualmente il 20% dei neuroni ad ogni passaggio per evitare la dipendenza tra singoli nodi. Inoltre è necessario specificare che la Loss è "pesata", il token 12 conta 1/5 rispetto agli altri, ciò è stato fatto per correggere il bias del modello verso la "nota ripetuta".

Bisogna poi dire che non viene usata la tecnica del padding e usa label smoothing 0.1. Il label smoothing è una tecnica di regolarizzazione del machine learning che modifica le etichette di addestramento rigide (hard targets) in etichette attenuate (soft targets) per impedire alle reti neurali di diventare eccessivamente sicure delle proprie predizioni, in questo caso assegnando una probabilità del 90% (0.9) alla classe corretta e distribuendo il restante 10% (0.1) in modo uniforme tra tutte le altre classi.

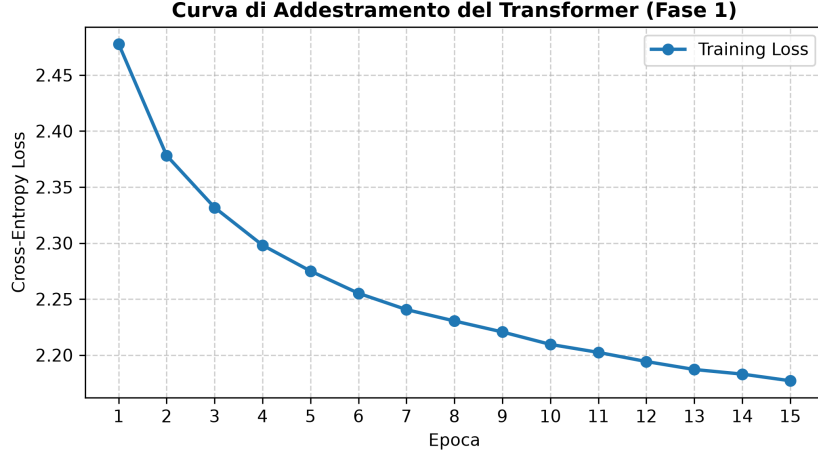


Figura 5.2: evoluzione della loss durante l'addestramento del modello

Una volta completato il training sarà possibile effettuare la generazione della melodia usando il modello creato.

Il campionamento probabilistico della melodia dal Music Transformer trasforma i valori grezzi calcolati dalla rete (logits) in una distribuzione di probabilità controllata. Il processo avviene in tre fasi sequenziali per selezionare l'intervallo musicale successivo.

1. I logits grezzi z_i prodotti dall'ultimo strato lineare rappresentano i punteggi non normalizzati associati a ciascun token/intervallo. Applicando il parametro di Temperatura T ($T = 0.85$), i logits vengono riscaldati tramite divisione:

$$z'_i = \frac{z_i}{T}$$

Se $T < 1.0$ allora T comprime i logits rendendo i valori più alti nettamente dominanti. Dopo la normalizzazione Softmax, la distribuzione diventa più "appuntita", riducendo la casualità e favorendo intervalli musicali stabili e coerenti. Se invece $T > 1.0$ allora T appiattisce la distribuzione rendendo le probabilità dei vari intervalli più omogenee, incrementando la varietà ma rischiando di generare linee melodiche caotiche.

2. Dopo aver scalato i logits con la temperatura, viene applicata la funzione Softmax per ricavare la probabilità di ciascun intervallo $p_i = \text{Softmax}(z'_i)$. Il campionamento Nucleus (Top-p sampling) seleziona solo il sottoinsieme dinamico di token più plausibili fino a raggiungere una probabilità cumulata pari a $p = 0.90$, la selezione funziona in questo modo:

- (a) I token vengono ordinati in ordine decrescente di probabilità.
- (b) Si accumulano i valori di probabilità partendo dal più alto; non appena la somma raggiunge o supera la soglia $p = 0.90$, tutti i token rimanenti vengono azzerati impostando il loro logit a $-\infty$.
- (c) A differenza del Top-k (che mantiene un numero fisso di opzioni), il Top-p adatta la dimensione del sottoinsieme alla certezza del modello. Se la rete è molto sicura, il

nucleo conterrà pochi intervalli; se è incerta, il nucleo si espanderà per includere più alternative ragionevoli.

3. Sui token superstiti che rientrano nel nucleo del 90%, viene ri-calcolata la distribuzione di probabilità ed eseguito un campionamento stocastico mediante estrazione multinomiale

Questo approccio combinato previene due problemi comuni della generazione autonoma: evita che la rete scelga sempre l'intervallo più probabile (evitando melodie monotone ed uguali ad ogni esecuzione) e impedisce che vengano estratti intervalli totalmente improbabili o stonati presenti nella "coda lunga" della distribuzione.

5.3 Selezione Automatica degli Strumenti

Per quanto riguarda gli strumenti ci sono 2 possibilità:

- Li sceglie l'utente.
- Il modello li seleziona in automatico.

La selezione automatica funziona così: Invece di guardare solo Valenza/Arousal, si guarda il contenuto semantico del testo.

Usa un classificatore Zero-Shot multilingue per confrontare la poesia con 6 "atmosfera" candidate (natura/bosco, sacro/religioso, epico/eroico, triste/malinconico, gioioso/festoso, notturno/sognante), ciascuna già mappata a una coppia (melodia, armonia) di strumenti.

Viene usato NLP Zero-Shot (l'apprendimento zero-shot e il few-shot migliorano l'NLP utilizzando dati etichettati minimi) sul testo se l'inferenza va a buon fine, se non va a buon fine si usa la regola Valence/Arousal/Tenderness che si basa sul dividere le parole in semplici categorie come "triste e felice", vista prima, se neanche l'emozione è disponibile si ricadrà su ("piano", "strings").

Capitolo 6

Rendering audio

I dati ottenuti precedentemente vengono poi accumulati e passati a una funzione che si occuperà di costruire il file .mid, questi dati sono:

- I dati musicali: La melodia, l'armonia, le note del basso e le note dell'arpeggio.
- I dati di esecuzione: la velocità del brano (tempo) e i nomi degli strumenti assegnati a ciascuna voce.
- Il percorso di salvataggio del file finale.

Dietro le quinte, il sistema convalida i nomi degli strumenti e li converte in numeri "Program Change" dello standard General MIDI (valori da 0 a 127). Questi valori vengono presi dal dizionario sintetico in modo che ogni player o soundfont sappia quale suono assegnare (ad esempio, il numero 0 per il pianoforte o il 40 per il violino).

6.1 Protocollo MIDI

Il MIDI, acronimo di Musical Instrument Digital Interface, è un protocollo standard che consente il collegamento e la comunicazione tra più strumenti musicali elettronici.

Essenzialmente, è un linguaggio informatico basato sullo scambio di messaggi e informazioni tra hardware.

All'interno del protocollo MIDI, gli strumenti possono essere di due tipi:

- Dispositivi di controllo (Controller) che servono a generare i messaggi MIDI, ad esempio una Tastiera MIDI muta o un Controller.
- Dispositivi sonori (Sound Module), che utilizzano i messaggi MIDI per produrre o modificare suoni. Questi possono essere Sintetizzatori, Drum Machine, Delay, Riverberi ed altri effetti.

Poiché il MIDI si esprime mediante i numeri, è necessario capire la convenzione adottata. Questa prevede che qualsiasi comando, come ad esempio il Volume, venga indicato con valori compresi tra 0 e 127.

Ciò significa che se volessi abbassare totalmente il volume del mio strumento, o synth, il messaggio che manderò conterrà come valore 0. Se invece volessi alzare il volume al massimo, il messaggio conterrà come valore 127.

Inoltre, il protocollo MIDI prevede 16 canali indipendenti per connettore. È possibile così inviare 16 messaggi diversi contemporaneamente tramite un'unico cavo. In questo modo possiamo suonare, ad esempio, più parti sovrapposte di un brano come Basso, Batteria e Piano.

Esistono due grandi famiglie di messaggi:

- Channel Message (messaggi di canale): sono messaggi indirizzati a canali specifici (da 1 a 16).
- System Common Message (messaggi di sistema): sono messaggi che non sono indirizzati a un canale specifico, e quindi sono ricevuti da tutti.

Fanno parte dei Channel Message: Note On, Note Off, Program Change (per cambiare preset), Control Change (messaggi concepiti per trasmettere valori che variano nel tempo), e molti altri.

Sono invece System Common Message: SPP (Song Pointer Position), Song Select Message, MTC (MIDI Time Code).

6.2 Costruzione del file

Utilizzando la libreria mido, il file viene costruito inviando pacchetti di protocollo MIDI nativo. Il file generato è multitraccia ed è composto in questo modo:

1. Traccia 0 → Non contiene note, ma le informazioni del brano, come la traccia del Tempo (BPM) calcolata in precedenza.
2. Traccia 1: Melodia (Canale 0) → Scrive la voce principale. Per questa traccia viene applicato un leggero "overlap" (sovrapposizione dei tick temporali) tra il rilascio di una nota e l'attacco della successiva, creando un effetto musicale "legato".
3. Traccia 2 - Armonia (Canale 1) → Scrive gli accordi. Per rendere l'esecuzione più realistica, il codice inserisce messaggi MIDI di Control Change 64 (l'equivalente del pedale del Sustain del pianoforte) all'inizio di ogni accordo per tenerlo premuto, rilasciandolo appena prima del cambio accordo.
4. Tracce 3 e 4 → Basso e Arpeggio (Canali 2 e 3): Vengono generate due tracce monofoniche aggiuntive per la linea di basso e per l'arpeggio.

6.3 Synth

Mentre la parte dell'algoritmo dedicata al protocollo MIDI si limita a compilare uno "spartito digitale", la parte di sintetizzazione si occupa di trasformare quelle istruzioni in veri e propri file audio stereo in formato WAV.

Sono supportati due approcci per generare la musica:

- Sintetizzatore Matematico Interno: Genera l'audio da zero scrivendo byte per byte le forme d'onda attraverso algoritmi, senza dipendere da campionamenti audio esterni.
- Sintetizzatore Esterno (FluidSynth): Delega la creazione dell'audio al programma di sistema FluidSynth. Questo metodo necessita del file MIDI e di un file SoundFont esterno (una libreria di suoni campionati reali) per esportare l'audio.

6.3.1 Sintetizzatore matematico interno

Ogni nota musicale viene tradotta dalla sua notazione MIDI in una frequenza espressa in Hertz. Per farlo viene applicata la seguente formula:

$$f = 440.0 \cdot 2^{(p-69)/12.0}$$

dove p è il valore numerico (pitch) della nota MIDI.

Successivamente viene generato il suono in base allo strumento scelto miscelando onde sinusoidali e armoniche. Su ogni strumento viene applicato un "involuppo ADSR" (Attacco, Decadimento, Sostegno, Rilascio). L'ADSR modella il volume nel tempo (es. un basso decade dolcemente, un organo mantiene un sostegno costante finché il tasto non viene rilasciato).

Una volta generate le singole onde sonore, il modulo unisce le quattro voci in un unico spazio stereo (panoramica), bilanciandone matematicamente i volumi:

- Melodia (Lead): Posizionata esattamente al centro (pan 0.0) con un volume prominente (85%).
- Armonia (Pad): Spostata sul canale sinistro (pan -0.45) con un volume più morbido (45%).
- Basso: Fissato al centro a frequenze basse (volume 70%).
- Arpeggio: Spostato sul canale destro (pan 0.50, volume 50%) per controbilanciare l'armonia.

6.3.2 FluidSynth

FluidSynth è un sintetizzatore software open source che trasforma i dati MIDI in segnale audio in tempo reale, basato sulla tecnologia SoundFont 2 (file .sf2/.sf3). In pratica non ha bisogno di una scheda audio compatibile con SoundFont: fa tutto via software.

Il principio è semplice: FluidSynth riceve eventi MIDI (note on/off, pitch bend, controllo, ecc.) e li "suona" usando i campioni audio precaricati contenuti in un file SoundFont, che definisce i suoni di tutti gli strumenti. La sintesi avviene in tempo reale e l'audio viene inviato a un dispositivo di uscita (ALSA, PulseAudio, PipeWire, JACK) oppure salvato su file.

FluidSynth implementa il modello di sintesi definito dalla specifica SoundFont 2. Non è un algoritmo "unico", ma una catena di elaborazione che parte dal campione audio registrato e lo modella in tempo reale. Ecco come funziona nel dettaglio.

La specifica SF2 definisce un modello composto da quattro blocchi principali:

- Un oscillatore wavetable (riproduzione del campione audio).
- Un filtro passa-basso dinamico.
- Un amplificatore con inviluppo (envelope).
- Dei send programmabili verso pan, riverbero e chorus.

Quando arriva un evento MIDI "note on", FluidSynth alloca una voce (voice) e la fa passare attraverso questa pipeline:

1. Wavetable oscillator: legge il campione PCM dal SoundFont e lo riproduce. La frequenza di riproduzione viene scalata per ottenere l'altezza desiderata (pitch shifting), partendo dal sample rate originale, dalla nota campionata e dalla correzione di intonazione memorizzati nel file.
2. Filtro passa-basso: un filtro risonante (con cutoff `initialFilterFc` e risonanza `initialFilterQ`) che modella il timbro, attenuando le armoniche.
3. Amplificatore con inviluppo: controlla il volume nel tempo tramite una curva ADSR (attack, decay, sustain, release), con parametri come `attackVolEnv`, `decayVolEnv`, `sustainVolEnv`, `releaseVolEnv`.
4. Modulazione: due LFO (uno per il vibrato `vibLfoToPitch`, uno per il filtro/volume `modLfoToFilterFc`) e due inviluppi di modulazione (`modEnvToPitch`, `modEnvToFilterFc`) che possono modificare pitch, cutoff del filtro e ampiezza in modo dinamico.
5. Routing finale: la voce viene posizionata nel campo stereo (pan) e inviata in quantità controllata ai bus di riverbero e chorus (`reverbEffectsSend`, `chorusEffectsSend`).

A questo si aggiunge un motore di modulazione formato da due LFO (oscillatori a bassa frequenza) e due generatori di inviluppo, con i relativi amplificatori di instradamento.

La chiave del modello SF2 è la distinzione tra due tipi di parametri:

- Generatori: valori diretti in ingresso al modello di sintesi (tuning, cutoff del filtro, attenuazione, tempi di inviluppo, quantità di LFO applicate a pitch/filtro/ampiezza).

- Modulatori: collegamenti da una sorgente dinamica (come un MIDI Continuous Controller) a un generatore, permettendo il controllo in tempo reale dei parametri di sintesi.

Questi parametri sono organizzati gerarchicamente: un preset contiene più preset zone, che puntano a uno strumento; lo strumento ha instrument zone, ognuna associata a un singolo campione. I valori del preset si sommano a quelli dello strumento (i primi sono relativi, i secondi assoluti).

Capitolo 7

Conclusioni

7.1 Valutazione finale

Il risultato ottenuto risulta soddisfacente, il programma sviluppato riesce a raggiungere gli obiettivi che si era posti.

Il programma ottenuto riesce a generare della musica coerente dato un prompt, inoltre il ritmo della canzone copre fedelmente quello degli accenti testo. Detto questo il progetto ha ancora margini di miglioramento.

7.2 Sviluppi futuri

Questo progetto lascia la porta aperta a numerosi sviluppi futuri:

- Si potrebbe migliorare il modello di generazione musicale, magari utilizzando un modello più complesso o addestrato su un dataset più ampio.
- Si potrebbe implementare oltre all'Italiano anche altre lingue, come l'inglese o lo spagnolo, per rendere il sistema più versatile.
- Si potrebbe aggiungere la possibilità di generare oltre alla musica anche una voce che canta il testo.

Capitolo 8

"Da rivedere"

8.1 Approccio generale

In questo capitolo ci si concentrerà sulla prima parte del progetto, ovvero la lettura del testo dato in input e la successiva analisi prosodica e semantica. Dopo si procederà con l'individuazione delle sillabe e degli accenti. Successivamente sarà fatta anche un'analisi emotiva per attribuire una melodia consona con il significato del testo e infine sarà prodotto lo scheletro ritmico.

8.2 Analisi Prosodica

Con analisi prosodica si intende è la parte della linguistica che studia l'intonazione, il ritmo (isocronia), la durata (quantità) e l'accento del linguaggio parlato.

In questo progetto ovviamente il linguaggio di riferimento sarà l'Italiano e come spiegato nel precedente capitolo i dataset importati per tale analisi sono Q2Stress e PhonItaliaR.

La pipeline di lavoro per questa parte sarà la seguente: Si prenderà il verso, dal verso saranno estratte le sillabe da cui poi si troveranno pattern accentuativo e alla fine sarà estratto il ritmo.

8.3 Librerie usate

8.3.1 re

Questa libreria offre delle operazioni svolgibili sulle espressioni regolari.

Un'espressione regolare specifica un set di stringhe che combacia con essa. Le funzioni di questa libreria permettono di verificare se una particolare stringa combacia con una particolare espressione regolare. Le espressioni regolari possono essere concatenate per formare nuove espressioni regolari; se A e B sono entrambe espressioni regolari, allora AB è anch'essa un'espressione regolare. In generale, se una stringa p corrisponde ad A e un'altra stringa q corrisponde a B, la stringa pq corrisponderà ad AB. Questo vale a meno che A o B contengano

operazioni a bassa priorità; condizioni ai confini tra A e B; o riferimenti a gruppi numerati. Fonte:<https://docs.python.org/3/library/re.html>

Nel codice implementato la libreria `re` serve per usare la funzione `re.sub` che rimuove tutti i caratteri di punteggiatura da una parola mantenendo solo le lettere e i caratteri accentati.

8.3.2 UnicodeData

La libreria dà accesso all'Unicode Character Database (UCD) che definisce le proprietà dei caratteri di tutti i caratteri Unicode.

Fonte: <https://docs.python.org/3/library/unicodedata.html>

Viene importata per gestire correttamente la codifica dei caratteri speciali e accentati. In particolare viene usato il metodo `lookup` che cerca un carattere dal nome. Se un carattere con tale nome viene trovato ritorna il carattere corrispondente, altrimenti viene lanciato un `Keyerror`.

8.3.3 Pyphen

Libreria Python per la sillabazione basata sui dizionari e sugli algoritmi di sillabazione di TeX (come quelli usati da OpenOffice/LibreOffice).

Nel codice viene usata come meccanismo di ripiego (`fallback`) o di confronto per la sillabazione dell'italiano (`it_IT`).

8.4 Algoritmo

L'algoritmo esegue l'analisi prosodica e ritmica di un testo poetico italiano trasformando ogni verso in una sequenza di impulsi temporali. Il processo si articola in quattro fasi consecutive.

8.4.1 Analisi prosodica

Per analizzare la prosodia di un testo, l'algoritmo dedicato procede al caricamento di due dataset: `Q2Stress` e `phonItaliaR`. Il primo viene utilizzato per addestrare il classificatore di accenti, il secondo per ampliare il training set. Inoltre, viene caricato anche il modello linguistico `spaCy` per l'analisi linguistica del testo (serve a tokenizzare il testo?). Le informazioni estratte dai dataset per la costruzione delle feature sono le parole annotate, la posizione dell'accento e il numero di sillabe della parola. Per ogni parola vengono estratte le seguenti feature:

1. Lunghezza della parola
2. Numero di vocali
3. Numero di consonanti
4. Rapporto vocali/consonanti

5. Numero di sillabe
6. Finale vocalico
7. Finale consonantico
8. Presenza di sequenze vocaliche (...)
9. Presenza di sequenze con t (...)
10. Suffissi finali (...)

In tutto sono 24 e servono per il calcolo della posizione dell'accento tonico per ogni parola del testo di input.

Queste caratteristiche sono utilizzate per addestrare il classificatore **Random Forest**. È un modello di Machine Learning supervisionato basato sugli alberi decisionali. Dopo l'addestramento sui dataset, impara a predire la posizione dell'accento tonico della parola categorizzandola in una delle seguenti categorie:

1. ultima sillaba
2. penultima sillaba
3. antepenultima sillaba
4. preantepenultima sillaba

In seguito, si passa all'acquisizione e alla normalizzazione del testo. Qui vengono eliminati eventuali problemi di codifica, uniformati gli spazi e rimossi gli elementi non necessari all'analisi (...). Il testo viene poi elaborato tramite spaCy, il quale per ogni parola ottiene:

1. token
2. lemma
3. categoria grammaticale (POS)
4. relazioni sintattiche
5. posizione nella frase
6. punteggiatura circostante

Prima di passare al Random Forest, avviene il conteggio delle sillabe grazie ai dati annotati nei dataset, Pyphen e un algoritmo euristico di fallback (...).

Infine, tutte queste informazioni vengono trasformate in una sequenza di impulsi ritmici. Ogni sillaba viene classificata come:

1. strong: accentata, dove sarà posizionata la nota dominante (corrispondente a un hit di batteria?)
2. weak: non accentata, dove sarà posizionata la stessa nota più lieve (hat?)

Inoltre, la punteggiatura generale aggiunge pause brevi e lunghe.

Infine, il sistema produce:

1. CSV analitico, il quale per ogni parola contiene l'accento predetto, il numero di sillabe, informazioni grammaticali e sintattiche
2. CSV ritmico che contiene la sequenza di impulsi forti, deboli e pause.
3. schema ritmico MIDI (opzionale?) utilizzabile nelle seguenti fasi di generazione musicale.

8.4.2 Produzione dello schema ritmico

(forse si può approfondire... togliendo la parte finale della sezione di sopra)

8.4.3 Scelta degli strumenti

8.4.4 Produzione della melodia

8.5 Analisi emotiva

Come prima cosa viene importato un dataset che sarà utilizzato per definire lo spettro emotivo della parola.

8.6 Librerie

8.6.1 re

Anche in questo algoritmo viene usata la libreria `re` già descritta precedentemente. In particolare viene usata in una funzione che si occupa di normalizzare una parola, ovvero la trasforma minuscola, senza punteggiatura, ciò è necessario perchè prima di iniziare con l'algoritmo è necessario controllare che la parola sia formattata correttamente.

Per implementare questa funzione vengono usate le funzioni `.lower` che come deducibile dal nome rende tutte le lettere minuscole e `.sub` già incontrata in precedenza.

Inoltre viene usata la funzione `.findall` che permette di ottenere gruppi di stringhe con pattern simili per la tokenizzazione

8.7 Algoritmo

Il cuore dell'algoritmo si trova nella funzione `analyze_emotions` che non classifica le parole in semplici categorie (come "triste" o "felice"), ma le mappa su tre dimensioni psicologiche continue:

- Valence (Valenza): Indica se un'emozione è positiva o negativa. Va da -1.0 (estremamente negativo, es. abominio) a +1.0 (estremamente positivo, es. felicità).
- Arousal (Attivazione): Misura l'intensità o il livello di eccitazione dell'emozione. Va da 0.0 (calma/letargia, es. pace) a +1.0 (alta eccitazione/tensione, es. terrore o vivacissimo).
- Tenderness (Tenerezza): Misura il grado di dolcezza o affetto. I valori positivi indicano tenerezza (es. amore = 1.0), mentre i valori bassi o negativi indicano durezza o brutalità (es. brutale = -0.6).

La funzione procede nel seguente modo:

1. Viene eseguita la "Tokenizzazione", il testo in ingresso viene suddiviso in singole parole usando un'espressione regolare.
2. Viene eseguita l'estrazione dei punteggi per ogni parola. La parola viene normalizzata, l'algoritmo controlla se la parola normalizzata esiste nel database di riferimento, se c'è un match (corrispondenza), estrae i tre valori (valenza, attivazione, tenerezza) e li salva in tre liste separate. La parola stessa viene aggiunta a una lista se la parola non è nel dizionario (es. articoli, congiunzioni o parole non censite), viene semplicemente ignorata.
3. Viene effettuato il calcolo della media dopo aver analizzato tutte le parole, l'algoritmo calcola la media aritmetica dei punteggi trovati. Questo rappresenta il "tono emotivo generale" della frase. Se nessuna parola del testo è presente nel dizionario, l'algoritmo restituisce valori neutri ovvero valence 0.0 (neutro), arousal 0.5 (attivazione media) e tenderness: 0.0 (neutro).
4. Viene calcolato il risultato finale: Viene Restituito un dizionario contenente le medie calcolate e la lista delle parole riconosciute.

Appendice A

Bibliografia e sitografia

A.1 Esempio di appendice

In appendice si possono inserire dettagli secondari, dimostrazioni o dati supplementari. Ad esempio, la seguente identità è utile in numerosi contesti:

$$\sum_{k=0}^n \binom{n}{k} = 2^n. \tag{A.1}$$

Questa forma può essere usata per riportare risultati accessori senza interrompere il flusso principale dell'articolo.